5. Collections

More Data Types

ISTITUTO TECNOLOGICO SUPERIORE ACADEMY - INFORMATION AND COMMUNICATIONS TECHNOLOGY
its
ICTAcademy
INFORMATION AND COMMUNICATIONS TECHNOLOGY

What are Collections?

Remember our primitive data types? There are more complex ones!

List	[]	`[1, 2, 5, 0, 4712]` or `["a", 5, "cde"]`
Set	{}	`{1, 2, 5, 0, 4712}` or `{"a", 5, "cde"}`
Dictionary	{}	`{"x":1, "y":2, "z":3}`

EXOTIC

Tuple	()	`(1,8)` or `("a", "b")` or `(0.5, "x")`

ICTAcademy
INFORMATION AND COMMUNICATIONS TECHNOLOGY

Lists

```
list1=[1,2,3,4,5]

list2=['A', 'B', 'C']

list3=["Hello", 1, True, 40.22]
```

ISTITUTO TECNOLOGICO SUPERIORE ACADEMY - INFORMATION AND COMMUNICATIONS TECHNOLOGY
its
ICTAcademy
INFORMATION AND COMMUNICATIONS TECHNOLOGY

Lists

```
list4=[1, [2,3,4], 5,6] #nested list [2,3,4] is a list inside list4

#display a list:
print(*list1) #display the list: 1 2 3 4 5

print(*list1, sep=", ") #display the list: [1,2,3,4,5]
```

ISTITUTO TECNOLOGICO SUPERIORE ACADEMY - INFORMATION AND COMMUNICATIONS TECHNOLOGY
its ICTAcademy
INFORMATION AND COMMUNICATIONS TECHNOLOGY

Working with Lists: Indexing

Indexing

```
mylist = ["Alice", "Bob", "Charlie"]
```

index = 0 1 2

We can use an **element's index** to **refer to it**!

```
print(mylist[0])
```

```
>>> "Alice"
```

ICTAcademy
INFORMATION AND COMMUNICATIONS TECHNOLOGY

Working with Lists: Indexing

```
mylist = [“Alice”, “Bob”, “Charlie”]
```

index	=	0	1	2
reverse index	=	-3	-2	-1

```
print(mylist[-1])
```

```
>>> “Charlie”
```

ISTITUTO TECNOLOGICO SUPERIORE ACADEMY - INFORMATION AND COMMUNICATIONS TECHNOLOGY
its ICTAcademy
INFORMATION AND COMMUNICATIONS TECHNOLOGY

Working with Lists: Using Indices

Now that we can specify our elements, we have more options to work with!

```
mylist = ["Alice", "Bob", "Chalie"]
```

We made a typo with "Chalie". How can we fix it?

```
mylist[-1] = "Charlie" or
mylist[2] = "Charlie"

print(mylist)
>>> ["Alice", "Bob", "Charlie"]
```

ICTAcademy
INFORMATION AND COMMUNICATIONS TECHNOLOGY

Lists

- **ordered**: the list holds the elements in the order you add them

```
mylist = [2, 5, 3]
mylist.append(19)
print(mylist)
>>> [2, 5, 3, 19]
```

- **mutable**: you can modify the list
- **allows duplicates**:

```
[2, 5, 2]                    is valid
["x", "x", 3]                is valid
```

ISTITUTO TECNOLOGICO SUPERIORE ACADEMY - INFORMATION AND COMMUNICATIONS TECHNOLOGY
its ICTAcademy
INFORMATION AND COMMUNICATIONS TECHNOLOGY

Working with Lists

Adding elements to a list: **`.append(element)`**

```
mylist = [“Alice”, “Bob”]
mylist.append(“Charlie”)
print(mylist)
>>> [“Alice”, “Bob”, “Charlie”]
```

Adding elements to a list: **`.insert(index, element)`**

```
mylist.insert(2, “Bob2.0”)
print(mylist)
>>> [“Alice”, “Bob”, “Bob2.0”, “Charlie”]
```

ISTITUTO TECNOLOGICO SUPERIORE ACADEMY - INFORMATION AND COMMUNICATIONS TECHNOLOGY
its ICTAcademy
INFORMATION AND COMMUNICATIONS TECHNOLOGY

Working with Lists

Removing elements from a list: **`.pop(index)`**

```
mylist = [“Alice”, “Bob”, “Bob2.0”, “Charlie”]
mylist.pop(2)
print(mylist)
>>> [“Alice”, “Bob”, “Charlie”]
```

Removing elements from a list: **`.remove(element)`**

```
mylist.remove(“Bob”)
print(mylist)
>>> [“Alice”, “Charlie”]
```

ISTITUTO TECNOLOGICO SUPERIORE ACADEMY - INFORMATION AND COMMUNICATIONS TECHNOLOGY
its ICTAcademy
INFORMATION AND COMMUNICATIONS TECHNOLOGY

Working with Lists

Adding elements to a list: **`.extend(list)`**

```
mylist = [“Alice”, “Charlie”]
mylist.extend([“Dolphin”, “Elephant”])
print(mylist)
>>> [“Alice”, “Charlie”, “Dolphin”, “Elephant”]
```

Delete a list: **`del`**

```
del mylist
#print(mylist)
```

its ICTAcademy
INFORMATION AND COMMUNICATIONS TECHNOLOGY

Working with Lists: Index Ranges

Indices can also specify a range of elements.
General syntax: `[start:end]`

```
mylist = [“apple”, “cookie”, “tea”, “coffee”]
```

We want to get the first two elements of the list. How?

```
print(mylist[0:2])
>>> [“apple”, “cookie”]
```

NOTE

The end index is excluded!
`[0:2]` -> returns elements at index `0` and `1`, but `2` is excluded.

ICTAcademy
INFORMATION AND COMMUNICATIONS TECHNOLOGY

Working with Lists: Index Ranges

Let's make it a little more difficult…

```
mylist = ["apple", "cookie", "tea", "coffee"]


print(mylist[1:]) #displays all the element of the list, starting from the index 1 to the
                   list end.
```

```
>>> ["cookie", "tea", "coffee"]
```

```
print(mylist[:1])#displays only the elements of the list strating from index 0
up               to index 1, excluding index 1
```

```
>>> ["apple"]
```

INFORMATION AND COMMUNICATIONS TECHNOLOGY

Working with Lists: Index Ranges

```
mylist = [“apple”, “cookie”, “tea”, “coffee”]

print(mylist[-2:])# displays the last two elements of the list
```

```
>>> [“tea”, “coffee”]
```

```
print(mylist[:]) #display a list copy.
```

```
>>> [“apple”, “cookie”, “tea”, “coffee”]
```

ISTITUTO TECNOLOGICO SUPERIORE ACADEMY - INFORMATION AND COMMUNICATIONS TECHNOLOGY
its
ICTAcademy
INFORMATION AND COMMUNICATIONS TECHNOLOGY

Exercise: Lists

1. Define a list called `alphabet`. It should contain the first 5 letters of the alphabet.
2. Define the variables `first_letter` and `last_letter`. Fill them accordingly.
3. Define the variable `first_three`. Use slicing to fill this variable.
4. Define the variable `last_three`. Use slicing to fill this variable.
5. Add the next 3 letters of the alphabet to our `alphabet`.
6. Now, update your variable `last_three`.
7. Delete the last letter from our `alphabet`.

Print out your variables whenever you need to!
You'll see what your code is doing.

INFORMATION AND COMMUNICATIONS TECHNOLOGY

Exercise: Lists, Solution

```
alphabet = [“a”, “b”, “c”, “d”, “e”]

first_letter = alphabet[0]
last_letter = alphabet[-1]

first_three = alphabet[:3]              alternative: alphabet[0:3]
last_three = alphabet[-3:]

alphabet.append(“f”)
alphabet.append(“g”)
alphabet.append(“h”)

last_three = alphabet[-3:]       >>> [“f”, “g”, “h”]

alphabet.remove(“h”)
```

ISTITUTO TECNOLOGICO SUPERIORE ACADEMY - INFORMATION AND COMMUNICATIONS TECHNOLOGY
its ICTAcademy
INFORMATION AND COMMUNICATIONS TECHNOLOGY

Sets

```
set_a={1,2,3,4,5}                #define a set

print(set_a)                     #display set: {1,2,3,4,5}
```

ISTITUTO TECNOLOGICO SUPERIORE ACADEMY - INFORMATION AND COMMUNICATIONS TECHNOLOGY
its
ICTAcademy
INFORMATION AND COMMUNICATIONS TECHNOLOGY

Sets

- **unordered**: there's no specific order to the elements

```
myset = {1, 7, 5}
print(myset)
>>> {1, 5, 7}
```

- **mutable**: you can modify the set
- **doesn't allow duplicates**:

```
myset = {1,1}
print(myset)
>>> {1}
```

ISTITUTO TECNOLOGICO SUPERIORE ACADEMY - INFORMATION AND COMMUNICATIONS TECHNOLOGY
its ICTAcademy
INFORMATION AND COMMUNICATIONS TECHNOLOGY

Working with Sets

Most important feature: **no duplicates!**

Say, we have a bunch of numbers, and we want to save them in a data structure.

```
1, 2, 1, 3, 2, 1, 3
```

But we want only unique numbers. What's the better choice, and what's the result of each of the following:

```
mylist =        [1, 2, 1, 3, 2, 1, 3]

myset =         {1, 2, 1, 3, 2, 1, 3}
```

```
>>> [1, 2, 1, 3, 2, 1, 3]

>>> {1, 2, 3}
```

ICTAcademy
INFORMATION AND COMMUNICATIONS TECHNOLOGY

Working with Sets

We can also make sets like this:

```
myset = set(…)
```

For example, to **convert** an **existing** list to a set:

```
list_with_duplicates = [1, 2, 1, 3]

no_duplicates = set(list_with_duplicates)

print(no_duplicates)
```

```
>>> {1, 2, 3}
```

NOTE

Converting an object from one type to another is called ***casting***. It also works for many other types.
E.g. `float("123") >>> 123.0`

its ICTAcademy
INFORMATION AND COMMUNICATIONS TECHNOLOGY

Working with Sets

Remember indexing from our lists? Try out if this works with sets, too.

- Declare a set
- Try printing the first element in the set
- Tell me what happens

```
>>> TypeError: 'set' object is not subscriptable
```

Why do you think that might be the case? Why can't we index sets?

→ *Remember: sets are **unordered**!*

ICTAcademy
INFORMATION AND COMMUNICATIONS TECHNOLOGY

Working with Sets

```
set_a={1,2,3,4,5}
set_b={4,5,6,7,8}
```

Adding elements to a set: **.add(element)**

```
set_a.add(6)          #add number 6 to set_a
print(set_a)          #output: {1,2,3,4,5,6}
```

Removing elements from a set: **.remove(element)**

```
set_a.remove(2)       #remove number 2 from set_a
print(set_a)          #output: {1,3,4,5,6}
```

ISTITUTO TECNOLOGICO SUPERIORE ACADEMY - INFORMATION AND COMMUNICATIONS TECHNOLOGY
its ICTAcademy
INFORMATION AND COMMUNICATIONS TECHNOLOGY

Working with Sets

```
set_a={1,2,3,4,5}
set_b={4,5,6,7,8}

print(set_a.union(set_b))     #output: {1,2,3,4,5,6,7,8}

print(set_a | set_b)          # symbol | (OR) means union. Output: {1,2,3,4,5,6,7,8}


print(set_a.intersection(set_b))     #output: {4,5}

print(set_a & set_b)                 # symbol & (AND) means intersection. Output: {4,5}



print(set_a.difference(set_b))       #output: {1,2,3}

print(set_a - set_b)                 # symbol - (MINUS) means difference. Output: {1,2,3}
```

its ICTAcademy
INFORMATION AND COMMUNICATIONS TECHNOLOGY

Dictionaries

With a Dictionary, we can collect this information in one place:

```
mydict = {key1:value1, key2:value2, key3:value3}
```

Dictionaries are unordered and mutable. However, **the keys need to be unique**!

key **value** **key** **value**

```
mydict={"ape":"Insetto dell’ordine imenotteri che produce miele",
        "balena":"enorme mammifero di oltre 20 m di lunghezza",
        "cane":"animale domestico appartenente alla famiglia dei Canidi"}
```

its ICTAcademy
INFORMATION AND COMMUNICATIONS TECHNOLOGY

Dictionaries… why is this cool again?

We don’t just save data in some data structure.
We even save **relations** between this data!

```
allAges = {“Alice”: 25, “Bob”: 30}
```

If we want to access the age of Alice now, we can:

```
alicesAge = allAges[“Alice”]

print(alicesAge)

>>> 25
```

ISTITUTO TECNOLOGICO SUPERIORE ACADEMY - INFORMATION AND COMMUNICATIONS TECHNOLOGY
its ICTAcademy
INFORMATION AND COMMUNICATIONS TECHNOLOGY

Dictionaries… why is this cool again?

```
allAges = {“Alice”: 25, “Bob”: 30}
```

Try printing the age of Bob!

```
bobsAge = allAges[“Bob”]
print(bobsAge)
>>> 30
```

By the way… we don’t need to declare `bobsAge` necessarily:

```
print(allAges[“Bob”])
>>> 30
```

ICTAcademy
INFORMATION AND COMMUNICATIONS TECHNOLOGY

Dictionaries: keys and values

We can access dictionaries through their keys. We can probably do the same with values:

```
allAges = {“Alice”: 25, “Bob”: 30}

print(allAges[25])
```

- Tell me what you think will happen
- Actually try it out

```
>>> KeyError: 25
```

Getting information from a dictionary doesn’t work both ways!

key -> value ✅ value -> key ❌

its ICTAcademy
INFORMATION AND COMMUNICATIONS TECHNOLOGY

Exercise: Dictionaries

We have a menu at a restaurant.

The available dishes are: pasta, pizza, salad, wine, water
Their prices are: 10.50, 9.00, 6.50, 4.00, 2.30

1. Save this information into a dictionary called `menu`.
2. From `menu`, access the prices of pasta and wine. Save them in separate variables.
3. Print the prices of pasta and wine.

its ICTAcademy
INFORMATION AND COMMUNICATIONS TECHNOLOGY

Exercise: Dictionaries, Solution

The available dishes are: pasta, pizza, salad, wine, water
Their prices are: 10.50, 9.00, 6.50, 4.00, 2.30

```
menu = {“pasta”: 10.50, “pizza”: 9.00, “salad”: 6.50,
            “wine”: 4.00, “water”: 2.30}

pasta_price = menu[“pasta”]
wine_price = menu[“wine”]

print(pasta_price)
print(wine_price)
```

ICTAcademy
INFORMATION AND COMMUNICATIONS TECHNOLOGY

Dictionaries: adding elements

We forgot dessert! Let's add cake to our menu.

```
menu = {"pasta": 10.50, "pizza": 9.00, "salad": 6.50,
            "wine": 4.00, "water": 2.30}
```

We can do it very straightforward:

```
menu["cake"] = 3.50
```

Try it, and print your menu again!

its ICTAcademy
INFORMATION AND COMMUNICATIONS TECHNOLOGY

Dictionaries: removing elements

To remove an entry, we can use `.pop()`

```
menu = {“pasta”: 10.50, “pizza”: 9.00, “salad”: 6.50,
            “wine”: 4.00, “water”: 2.30}
```

Try removing salad from our `menu`.

```
menu.pop(“salad”)
print(menu)
```

```
>>> {“pasta”: 10.50, “pizza”: 9.00, “wine”: 4.00,
     “water”: 2.30, “cake”: 3.50}
```

ICTAcademy
INFORMATION AND COMMUNICATIONS TECHNOLOGY

Dictionaries: modifying existing entries

Besides adding/deleting, we can change existing entries.

```
menu = {“pasta”: 10.50, “pizza”: 9.00, “wine”: 4.00,
        “water”: 2.30, “cake”: 3.50}
```

We’re feeling particularly nice, and want to lower the pizza price to 8.00.

Any idea how we might do that?

```
menu[“pizza”] = 8.00

print(menu)
>>> {“pasta”: 10.50, “pizza”: 8.00, “wine”: 4.00,
     “water”: 2.30, “cake”: 3.50}
```

ICTAcademy
INFORMATION AND COMMUNICATIONS TECHNOLOGY

Tuples

Tuples are a data type in Python used to store an **ordered sequence of elements**. They are very similar to lists, but with a key difference: **tuples are immutable**, meaning their content cannot be changed after they are created.

Key characteristics of tuples:

- **Ordered**: The elements have a specific order and can be accessed using an index.
- **Immutable**: Once created, you cannot add, remove, or modify elements.
- **Supports mixed types**: Tuples can hold elements of different data types (e.g., numbers, strings, other tuples, etc.).
- **Efficient**: Tuples are faster than lists for read operations and use less memory.

```
# Tuple with values
my_tuple = (1, 2, 3)

# Empty tuple
empty_tuple = ()

# Tuple with a single element (note the comma)
single_element_tuple = (5,)
```

its ICTAcademy
INFORMATION AND COMMUNICATIONS TECHNOLOGY

Differences between Tuples and Lists

Feature	Lists	Tuples
Syntax	Enclosed in square brackets `[ ]`	Enclosed in parentheses `( )`
Mutability	Mutable (can be modified)	Immutable (cannot be modified)
Speed	Slower for read operations	Faster for read operations
Typical use	For data that needs modification	For data that should remain unchanged
Operations	You can add, remove, or modify elements	You cannot modify elements after creation

Lists

```
my_list = [1, 2, 3]

# You can modify elements
my_list[0] = 10

print(my_list)  #Output: [10, 2, 3]
```

its ICTAcademy
INFORMATION AND COMMUNICATIONS TECHNOLOGY

Differences between Tuples and Lists

Feature	Lists	Tuples
Syntax	Enclosed in square brackets `[ ]`	Enclosed in parentheses `( )`
Mutability	Mutable (can be modified)	Immutable (cannot be modified)
Speed	Slower for read operations	Faster for read operations
Typical use	For data that needs modification	For data that should remain unchanged
Operations	You can add, remove, or modify elements	You cannot modify elements after creation

Tuples

```
my_tuple = (1, 2, 3)

#my_tuple[0] = 10
#This will raise an error, because tuples are immutable

print(my_tuple) #Output: (1, 2, 3)
```

its ICTAcademy
INFORMATION AND COMMUNICATIONS TECHNOLOGY

Tuples

Tuples support the same element access methods as lists:

```
my_tuple = (10, 20, 30)

# Accessing elements by index
print(my_tuple[0])  # Output: 10

# Slicing (extracting a sub-tuple)
print(my_tuple[1:])  # Output: (20, 30)
```

ISTITUTO TECNOLOGICO SUPERIORE ACADEMY - INFORMATION AND COMMUNICATIONS TECHNOLOGY
its ICTAcademy
INFORMATION AND COMMUNICATIONS TECHNOLOGY

Tuples

```
my_tuple=(1, "strings", 4.5, True)   #define a tuple

print(my_tuple[1])                   #displays: "strings"

print("\n" + str(type(my_tuple)))    #prove my_tuple is a tuple!

print(my_tuple.count("strings"))     #get the index of «strings» element

print(my_tuple.index(4.5))           #get the index of 4.5 element
```

its ICTAcademy
INFORMATION AND COMMUNICATIONS TECHNOLOGY